

Interval width and bootstrap iterations: polytomous items

Generalisation check for the follow-up to Johansson (2025)

Magnus Johansson

2026-07-29

Table of contents

Scope	1
Matched to the dichotomous study	1
A note on effect size	1
Design	1
Data generation	2
Run	3
Does $1 - w^k$ hold for polytomous items?	4
Detection and false positives (3 misfitting items)	5
p-value rules	5
Head-to-head at matched family-wise error	6

Scope

Companion to `infit_width_iterations.qmd`, which used 20 **dichotomous** items. Two claims established there are model-agnostic in principle but were only measured on dichotomous data:

1. The HDCl rule's family-wise error is approximately $1 - w^k$ (the Sidak relation), so `hdci_width` is the error-rate parameter.
2. The Westfall-Young FWER p-value matches or beats the width-based rule at equal error, and needs a smaller B because it estimates no extreme quantile.

This document repeats the design with **polytomous (PCM) items**.

Matched to the dichotomous study

Item locations are generated with the *same seed and call* as the dichotomous study (`set.seed(20250727); runif(20, -2, 2)`), so the 20 items sit at identical locations and the misfitting items are the same three — V8, V12, V11 at 0.04, -1 and -2 logits. Person locations, the inter-dimensional correlation (0.15) and the misfit mechanism are also unchanged. **The only difference from the dichotomous study is the response format**: four ordered categories (0-3) with thresholds spread +/- 1 logit around each item location, instead of a binary response.

Keeping $k = 20$ means this tests item *type* while holding item *count* fixed. The separate question of whether $1 - w^k$ tracks a different k is better answered with a cheap dichotomous run at $k = 10$ than by changing two things at once here.

Data are generated with the package's own PCM generator (`sim_partial_score()`), the same one `RMitemInfitCutoff()` uses internally; misfitting items are drawn from the second theta dimension, fitting items from the first.

A note on effect size

Polytomous items carry more information per item, so the same inter-dimensional correlation produces **much stronger** misfit than in the dichotomous study: at $n = 500$ the misfitting items show `infit` around 1.75-2.03 against 0.74-0.93 for fitting items. Detection therefore saturates, and the informative contrast between rules is on the false-positive side (the overfit cascade) rather than on detection. The primary question here — whether $1 - w^k$ holds under a true model — is unaffected, since it is measured at $k = 0$.

Design

- 20 polytomous items, 4 categories; locations and misfitting items identical to the dichotomous study
- n in {250, 1000}; B in {100, 400, 2000}; w in {.90, .95, .99, .995, .999}
- number of misfitting items in {0, 3}
- independent bootstrap per (`rep`, k , n , B) cell
- R2 starts at 100; runs are resumable, so raising it computes only the new replications

```
suppressMessages({
  pkgload::load_all("../", quiet = TRUE) # sim_partial_score() is internal
  library(ggdist)
  library(MASS)
```

```

library(parallel)
library(dplyr)
library(tidyr)
library(ggplot2)
})

NITEMS <- 20
NCAT <- 4
WIDTHS <- c(0.90, 0.95, 0.99, 0.995, 0.999)
NS <- c(250, 1000)
BS <- c(100, 400, 2000)
NMISFITS <- c(0, 3)
R2 <- 100 # raise and re-render to extend
NCORES <- 10
ALPHA <- 0.05
CHUNK <- 40
RES <- "infit_width_iterations_poly.rds"

```

```

run_resumable <- function(res_file, grid, keys, fn,
                          ncores = NCORES, chunk_size = CHUNK,
                          extra = list()) {
  key_of <- function(g) {
    if (nrow(g) == 0L) return(character(0))
    do.call(paste, c(unname(as.list(g[keys])), sep = "|"))
  }
  acc <- if (file.exists(res_file)) readRDS(res_file) else
    c(list(res = list(), grid = grid[0, , drop = FALSE], elapsed_min = 0), extra)
  todo <- which(!(key_of(grid) %in% key_of(acc$grid)))
  if (length(todo) == 0L) {
    message("cached: ", nrow(acc$grid), " cells, nothing to run"); return(acc)
  }
  message("running ", length(todo), " new cells (", nrow(acc$grid), " cached)")
  for (s in seq(1, length(todo), by = chunk_size)) {
    idx <- todo[s:min(s + chunk_size - 1L, length(todo))]
    t0 <- Sys.time()
    out <- mclapply(idx, function(i) fn(grid[i, , drop = FALSE])),
              mc.cores = ncores)
    acc$res <- c(acc$res, out)
    acc$grid <- rbind(acc$grid, grid[idx, , drop = FALSE])
    acc$elapsed_min <- acc$elapsed_min + as.numeric(Sys.time() - t0, "mins")
    acc <- utils::modifyList(acc, extra)
    saveRDS(acc, res_file)
    message(" checkpoint ", nrow(acc$grid), " / ", nrow(grid),
            " (", round(acc$elapsed_min, 1), " min)")
  }
  acc
}

```

Data generation

```

set.seed(20250727) # identical to the dichotomous study
LOCS <- runif(NITEMS, -2, 2)
MISFIT <- integer(0)
for (tg in c(0, -1, -2)) {
  MISFIT <- c(MISFIT, setdiff(order(abs(LOCS - tg)), MISFIT)[1])
}
DELTAS <- lapply(LOCS, function(L) L + seq(-1, 1, length.out = NCAT - 1))
SIGMA <- matrix(c(1.5^2, .15 * 1.5^2, .15 * 1.5^2, 1.5^2), 2, 2)

sim_poly <- function(n, k, seed) {
  set.seed(seed)
  th <- MASS::mvrnorm(n, mu = c(0, 0), Sigma = SIGMA)
}

```

```

mis <- MISFIT[seq_len(k)]
d <- matrix(0L, n, NITEMS)
fit_items <- setdiff(seq_len(NITEMS), mis)
if (length(fit_items)) d[, fit_items] <- sim_partial_score(DELTAS[fit_items], th[, 1])
if (length(mis)) d[, mis] <- sim_partial_score(DELTAS[mis], th[, 2])
d <- as.data.frame(d)
names(d) <- paste0("V", seq_len(NITEMS))
d
}

data.frame(item = paste0("V", MISFIT), intended = c(0, -1, -2),
          actual = round(LOCS[MISFIT], 2))

```

item	intended	actual
V8	0	0.04
V12	-1	-1.00
V11	-2	-2.00

```

bounds_at <- function(results, width, items) {
  vapply(items, function(it) {
    v <- results$InfitMSQ[results$Item == it]
    as.numeric(ggdist::hdci(v, .width = width))[1:2]
  }, numeric(2))
}

flag_code <- function(msq, lo, hi) ifelse(msq > hi, 2L, ifelse(msq < lo, 1L, 0L))

```

Run

```

one_rep <- function(g) {
  rep <- g$rep; k <- g$k; n <- g$n
  d <- sim_poly(n, k, seed = 8e7 + 1e6 * k + 1e3 * n + rep)
  items <- names(d)
  obs <- try(RMitemInfit(d, output = "dataframe"), silent = TRUE)
  if (inherits(obs, "try-error")) return(NULL)
  msq <- obs$Infit_MSQ[match(items, obs$Item)]
  hd <- list(); pv <- list()
  for (j in seq_along(BS)) {
    co <- try(RMitemInfitCutoff(d, iterations = BS[j], parallel = FALSE,
                               seed = rep * 100 + j, verbose = FALSE),
             silent = TRUE)
    if (inherits(co, "try-error")) next
    hd[[length(hd) + 1L]] <- do.call(rbind, lapply(WIDTHS, function(w) {
      b <- bounds_at(co$results, w, items)
      data.frame(B = BS[j], w = w, item = seq_along(items),
                 code = flag_code(msq, b[1, ], b[2, ]))
    })))
    fw <- try(suppressWarnings(
      RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
                  alpha = ALPHA, output = "dataframe")), silent = TRUE)
    bh <- try(suppressWarnings(
      RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fdr_bh",
                  alpha = ALPHA, output = "dataframe")), silent = TRUE)
    if (!inherits(fw, "try-error") && !inherits(bh, "try-error")) {
      i1 <- match(items, fw$Item); i2 <- match(items, bh$Item)
      pv[[length(pv) + 1L]] <- data.frame(
        B = BS[j], item = seq_along(items),
        p_marg = fw$p_infit[i1], padj_fwer = fw$padj_infit[i1],
        padj_bh = bh$padj_infit[i2])
    }
  }
}

```

```

  list(hdci = do.call(rbind, hd), pval = do.call(rbind, pv))
}

grid <- expand.grid(rep = 1:R2, n = NS, k = NMISFITS)
s <- run_resumable(RES, grid, c("rep", "k", "n"), one_rep,
  extra = list(misfit = MISFIT, locs = LOCS))

bind_part <- function(part) {
  do.call(rbind, lapply(seq_len(nrow(s$grid)), function(i) {
    r <- s$res[[i]][[part]]
    if (is.null(r)) return(NULL)
    cbind(rep = s$grid$rep[i], k = s$grid$k[i], n = s$grid$n[i], r)
  }))
}

tag <- function(x) x |>
  mutate(is_misfit = mapply(function(k, it) it %in% s$misfit[seq_len(k)], k, item))
H <- bind_part("hdci") |> tag() |>
  mutate(flag = as.integer(code > 0), w = sprintf("%.3f", w))
P <- bind_part("pval") |> tag()
cat(nrow(s$grid), "cells |", round(s$elapsed_min, 1), "min |",
  length(unique(s$grid$rep)), "reps\n")

```

400 cells | 282.9 min | 100 reps

Does $1 - w^k$ hold for polytomous items?

Predicted family-wise error at $k = 20$:

```

data.frame(w = sprintf("%.3f", WIDTHS),
  predicted = round(100 * (1 - WIDTHS^NITEMS), 1))

```

w	predicted
0.900	87.8
0.950	64.2
0.990	18.2
0.995	9.5
0.999	2.0

Measured under a true model:

```

H |>
  filter(k == 0) |>
  group_by(w, B, n, rep) |>
  summarise(any = as.integer(sum(flag) > 0), .groups = "drop") |>
  group_by(w, B, n) |>
  summarise(v = 100 * mean(any), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(w, B)

```

w	B	n=250	n=1000
0.900	100	93	98
0.900	400	90	90
0.900	2000	90	88
0.950	100	80	74

w	B	n=250	n=1000
0.950	400	74	72
0.950	2000	67	58
0.990	100	41	38
0.990	400	24	24
0.990	2000	22	20
0.995	100	38	37
0.995	400	15	18
0.995	2000	7	11
0.999	100	38	37
0.999	400	6	7
0.999	2000	2	1

Detection and false positives (3 misfitting items)

```
H |>
  filter(k == 3) |>
  mutate(type = ifelse(is_misfit, "det", "fp")) |>
  group_by(w, B, n, type) |>
  summarise(v = 100 * mean(flag), .groups = "drop") |>
  pivot_wider(names_from = c(type, n), values_from = v) |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(w, B)
```

w	B	det_250	fp_250	det_1000	fp_1000
0.900	100	100	75.9	100	99.9
0.900	400	100	75.2	100	99.8
0.900	2000	100	73.5	100	99.9
0.950	100	100	66.9	100	99.6
0.950	400	100	65.0	100	99.6
0.950	2000	100	63.4	100	99.7
0.990	100	100	48.6	100	98.5
0.990	400	100	39.6	100	98.2
0.990	2000	100	38.1	100	98.2
0.995	100	100	44.6	100	97.3
0.995	400	100	32.6	100	97.1
0.995	2000	100	30.6	100	97.2
0.999	100	100	44.6	100	97.3
0.999	400	100	26.7	100	94.4
0.999	2000	100	17.2	100	91.9

p-value rules

```
PL <- P |>
  mutate(fwer = as.integer(padj_fwer < ALPHA),
         bh = as.integer(padj_bh < ALPHA),
```

```

    uncorr = as.integer(p_marg < ALPHA)) |>
pivot_longer(c(fwer, bh, uncorr), names_to = "rule", values_to = "flag")

PL |>
  filter(k == 0) |>
  group_by(rule, B, n, rep) |>
  summarise(any = as.integer(sum(flag, na.rm = TRUE) > 0), .groups = "drop") |>
  group_by(rule, B, n) |>
  summarise(v = 100 * mean(any), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(rule, B)

```

rule	B	n=250	n=1000
bh	100	0	0
bh	400	2	3
bh	2000	1	5
fwer	100	5	6
fwer	400	5	4
fwer	2000	4	4
uncorr	100	70	62
uncorr	400	71	62
uncorr	2000	66	58

```

PL |>
  filter(k == 3) |>
  mutate(type = ifelse(is_misfit, "det", "fp")) |>
  group_by(rule, B, n, type) |>
  summarise(v = 100 * mean(flag, na.rm = TRUE), .groups = "drop") |>
  pivot_wider(names_from = c(type, n), values_from = v) |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(rule, B)

```

rule	B	det_250	fp_250	det_1000	fp_1000
bh	100	99	44.2	100	99.6
bh	400	100	49.4	100	99.6
bh	2000	100	51.4	100	99.8
fwer	100	100	19.5	100	99.5
fwer	400	100	17.5	100	99.6
fwer	2000	100	16.5	100	99.8
uncorr	100	100	58.9	100	99.6
uncorr	400	100	61.6	100	99.6
uncorr	2000	100	61.9	100	99.8

Head-to-head at matched family-wise error

```

bind_rows(
  H |> filter(k == 3, B == 2000, w == "0.999") |>
  group_by(n) |>
  summarise(rule = "HDCI w=.999 B=2000",

```

```

    det = round(100 * mean(flag[is_misfit]), 1),
    fp = round(100 * mean(flag[!is_misfit]), 1), .groups = "drop"),
PL |> filter(k == 3, B == 2000, rule == "fwer") |>
  group_by(n) |>
  summarise(rule = "FWER B=2000",
    det = round(100 * mean(flag[is_misfit], na.rm = TRUE), 1),
    fp = round(100 * mean(flag[!is_misfit], na.rm = TRUE), 1),
    .groups = "drop")
) |> arrange(n, rule)

```

n	rule	det	fp
250	FWER B=2000	100	16.5
250	HDCI w=.999 B=2000	100	17.2
1000	FWER B=2000	100	99.8
1000	HDCI w=.999 B=2000	100	91.9